

Health Insurance Multi-Agent System — Architecture & Specification

Comprehensive Technical Documentation: Inputs, Components, Outputs, & Data Formats

1. System Purpose & High-Level Architecture

This project is an enterprise-grade Multi-Agent GenAI System designed for Indian Health Insurance recommendations. It accepts natural language user questions regarding policy feature comparisons, IRDAI risk metrics, and detailed policy brochure text. A **ReAct Master Dispatcher** dynamically orchestrates three specialized tools, maintaining short-term hybrid conversation memory and validating final output accuracy against automated benchmark suites.

Layer	Technology / Implementation
Frontend UI	Streamlit (Session state, chat bubbles, tool badges)
REST API	FastAPI (Asynchronous execution, CORS, Pydantic data validation)
Orchestration	LangChain ReAct Agent Executor backed by Groq (Llama-3.3-70B)
Structured DB	MongoDB Atlas M0 Free Tier (Collection: insurance_db.health_insurance)
Vector Store / RAG	FAISS + HuggingFace sentence-transformers (all-MiniLM-L6-v2) + Web Search Fallback
Risk & Metrics	Versioned IRDAI FY2024 JSON + RapidFuzz Centralized Matching
Evaluation Suite	Automated 5-query benchmark accuracy testing script (backend/eval_suite.py)

2. Component Specifications (Inputs, Processing, Outputs)

Component 1: Streamlit Frontend (frontend/streamlit_app.py)

Input: User query string from chat input box + Session ID stored in st.session_state.

Processing: Sends HTTP POST request to FastAPI backend (/chat). Renders assistant chat bubbles and displays a styled tool-badge showing exact tool name(s) called.

Output Format: Interactive Web Chat UI rendering Markdown text and tool execution badges.

Component 2: FastAPI Backend (backend/app/main.py)

Input: JSON Request Payload: {"session_id": str, "query": str}

Processing: Validates inputs using Pydantic models (ChatRequest). Passes query to Master Dispatcher, applies Guardrails validation, and returns structured response.

Output Format: JSON Response Payload:
{ "session_id": str, "query": str, "answer": str, "tools_used": list[str] }

Component 3: ReAct Master Dispatcher (backend/app/dispatcher.py)

Input: session_id: str, query: str

Processing: Loads past chat history from Memory Manager. Constructs ReAct prompt with 3 tools (compare_policies, insurer_financial_risk, policy_document_qa). Uses Groq deep_model (Llama-3.3-70B) for multi-step reasoning. Tracks executed tool names.

Output Format: Python Dict: {"answer": str, "tools_used": list[str]}

Component 4: Text-to-MongoDB Compare Tool (backend/app/tools/compare_tool.py)

Input: Natural language comparison query string.

Processing: Prompts deep_model to generate a single-line PyMongo find() query. Verifies query safety via is_query_safe() (blocking write/delete keywords). Executes query against MongoDB Atlas collection.

Output Format: Formatted Markdown comparison text detailing ratings ('good'/'average'/'bad') and feature descriptions across requested policies.

Component 5: Dynamic Insurer Risk Tool (backend/app/tools/risk_tool.py)

Input: Natural language query containing an insurer/provider name.

Processing: Extracts provider name using Centralized Fuzzy Matcher. Reads versioned IRDAI data from data/risk_data.json. Enforces data freshness disclaimer.

Output Format: Structured string containing Provider, Claim Settlement Ratio (CSR), Incurred Claim Ratio (ICR), Solvency Ratio, and explicit 'IRDAI FY2024' report disclaimer.

Component 6: Centralized Fuzzy Matcher (backend/app/tools/fuzzy_matcher.py)

Input: query: str, score_threshold: int = 50

Processing: Checks direct aliases (e.g. 'max bupa' -> 'niva-bupa', 'religare' -> 'care-health'). Uses rapidfuzz.process.extractOne partial ratio against canonical registries in data/policy_names.py.

Output Format: Tuple: (canonical_provider_name: str, match_score: int)

Component 7: FAISS Vector Index & PDF Ingest Pipeline (backend/app/ingestion/pdf_ingest.py)

Input: pdf_source (URL or local path), provider: str, policy_name: str

Processing: Extracts text with pdfplumber, splits into 500-char chunks (overlap=50), embeds with local sentence-transformers/all-MiniLM-L6-v2 model on CPU. Loads existing FAISS index and appends without overwriting.

Output Format: Boolean status (True/False) + Persistent index stored in faiss_index/ directory.

Component 8: Web Search Agent (backend/app/ingestion/web_search_agent.py)

Input: policy_name: str

Processing: Executes Tavily Search for policy wording/brochure PDF. Prompts Groq light_model to evaluate search results and isolate direct PDF download link.

Output Format: Direct PDF URL string (e.g., https://.../policy_wording.pdf) or empty string.

Component 9: RAG Tool (backend/app/tools/rag_tool.py)

Input: Natural language policy query.

Processing: Performs similarity search on FAISS vector store. If empty, triggers Web Search Agent -> PDF Ingest pipeline -> Re-searches FAISS. Synthesizes final answer using Groq light_model.

Output Format: Detailed synthesized answer string citing retrieved document context.

Component 10: Hybrid Session Memory Manager (backend/app/memory.py)

Input: session_id: str, new message turn context.

Processing: Maintains ConversationSummaryBufferMemory(max_token_limit=300) per session. Keeps recent turns verbatim and uses light_model to summarize older turns.

Output Format: LangChain chat history message objects.

Component 11: Output Validation Guardrails (backend/app/guardrails.py)

Input: query: str, raw_answer: str

Processing: Checks non-emptiness, validates response formatting, and prevents hallucinated output claims.

Output Format: Clean validated response string.

Component 12: Evaluation Benchmark Suite (backend/eval_suite.py)

Input: Array of 5 benchmark test cases containing expected tools and required keywords.

Processing: Executes queries through handle_query, verifies tool routing accuracy and answer keyword presence.

Output Format: Accuracy report string (e.g., Overall Accuracy: 5/5 Passed (100.0% Correct)).

3. Core Data Schemas & Formats

MongoDB Policy Document Schema (insurance db.health insurance):

```
{
  "provider": "hdfc-ergo",
  "insurance_name": "hdfc-ergo-optima-secure",
  "description": "Flagship product offering 4X coverage benefit.",
  "features": {
    "maternity-cover": {"rating": "bad", "details": "Not covered in Optima Secure"},
    "room-rent-limit": {"rating": "good", "details": "No room rent capping"},
    "waiting-period": {"rating": "good", "details": "30 days initial, 36 months PED"}
  }
}
```

Risk Data JSON Schema (data/risk_data.json):

```
{
  "as_of_year": 2024,
  "source": "IRDAI Annual Report FY2023-24",
  "data": {
    "icici-lombard": {
      "display_name": "ICICI Lombard General Insurance",
      "CSR": "98.6%", "ICR": "78.4%", "solvency_ratio": "2.62",
      "summary": "Strong claim settlement ratio (98.6%) and healthy solvency."
    }
  }
}
```